ELFC-GPIO

A hardware access library for the ElfC C compiler for the CDP1802. It tests the EF lines, reads and writes the I/O ports, waits for a time, counts the ticks of the real time clock, and talks to the devices on an I2C bus and on an SPI adapter.

The library makes no BIOS or operating system calls, and calls nothing in the ElfC runtime, so the same `gpio.lib` works in programs for both Elf/OS and ELF-DOS.

The functions are written in assembly, not in C with inline assembly. They take their arguments straight from the expression stack and have no stack frame, which keeps them small and fast.

Building

ElfC has to be on the path.

```
make                build lib/gpio.lib and the I2C drivers
make examples       build the example programs for Elf/OS
make install        copy the libraries and gpio.h into ElfC (default
/opt/elfc)
make clean          remove the generated files
```

Use `make install ELFC_DIR=folder` if ElfC is installed somewhere else, and `make -C examples TARGET_OS=elfdos` to build the examples for ELF-DOS.

Two settings are built into the library. Run `make clean` before changing them:

Setting	Default	Description
`EXP_PORT`	5	The port that selects the port group. It is 5 on the 1802/Mini, 1802/MAX and Super Elf, and 1 on most other systems. A program can change it with `set_exp_port`.
`FREQ_KHZ`	4000	The clock frequency the delay functions assume if the BIOS has not stored the real one.

```
make EXP_PORT=1 FREQ_KHZ=1790
```

Using the library

Include `gpio.h` in the program. The header file tells the linker to search `gpio.lib`, so no linker option is needed:

```
#include <gpio.h>
```

```
#define LEDS  PORT(0, 4)

int main(void) {
  byte count = 0;

  while (!ef4_asserted()) {
    out_port(LEDS, count++);
    delay_ms(250);
  }
  return 0;
}
```

```
elfc blink.c         for Elf/OS
elfc -E blink.c      for ELF-DOS
```

The linker looks for the libraries in the `lib` folder of ElfC and in the current folder. Without `make install`, copy `lib/gpio.lib` next to the program and include the header file by its path. A program that uses I2C needs the library of its I2C driver there as well.

Functions

Function	Description
`bool ef1_asserted(void)`	True if the EF1 line is asserted. Also `ef2_asserted`, `ef3_asserted` and `ef4_asserted`.
`void out_port(port_t port, byte value)`	Write a byte to a port.
`byte inp_port(port_t port)`	Read a byte from a port.
`void set_exp_port(byte port)`	Set the port that selects the port group, 1 to 7.
`void delay_ms(unsigned int ms)`	Wait for 0 to 65535 milliseconds.
`void delay_s(unsigned int s)`	Wait for 0 to 65535 seconds.
`void tick_start(port_t rtc)`	Start counting the ticks of the real time clock, 64 in a second.
`void tick_stop(void)`	Stop the ticks.
`byte ticks(void)`	The number of ticks so far.
`void tick_wait(byte n)`	Wait until `n` ticks after the wait before.
`bool i2c_init(port_t port)`	Set the port of the I2C bus and free the bus.

Function	Description
`bool i2c_clear(void)`	Free the bus from a device that holds the data line low.
`bool i2c_write(byte addr, const byte *data, unsigned int len)`	Send a message to a device.
`bool i2c_read(byte addr, byte *data, unsigned int len)`	Receive a message from a device.
`bool i2c_transfer(byte addr, const byte *wdata, unsigned int wlen, byte *rdata, unsigned int rlen)`	Send a message and, after a repeated start, receive one.
`void spi_init(port_t ctrl, port_t data)`	Set the control port and the data port of the SPI adapter.
`void spi_control(byte lines)`	Set the chip selects and the general outputs of the adapter.
`void spi_write(const byte *data, unsigned int len)`	Send bytes.
`void spi_read(byte *data, unsigned int len)`	Receive bytes.
`void spi_transfer(const byte *wdata, byte *rdata, unsigned int len)`	Send bytes and receive one for each of them.
`void spi_write_dma(const byte *data, unsigned int len)`	Send bytes with DMA.
`void spi_read_dma(byte *data, unsigned int len)`	Receive bytes with DMA.

`byte` is `unsigned char`, and `bool` comes from `stdbool.h`.

Ports

A port is a `port_t`, an `unsigned int` with the port group in the high byte and the port index, 1 to 7, in the low byte. The `PORT(group, index)` macro makes one:

```
#define UART_DATA  PORT(0, 6)
#define RTC        PORT(1, 3)
```

The group and the index are in one value because of how ElfC calls a function. Every argument is pushed as two bytes, whatever its type, so three `byte` arguments cost three pushes. A structure costs more still, as it is copied onto the stack by a runtime routine. With a constant group and index, `PORT` is a constant that is pushed once. These are the sizes of a call with a constant port and the value in a global variable:

`out_port` takes	Bytes for the call	Pushes
group, index, value	23	3
structure, value	26	2 and a structure copy
`port_t`, value	19	2

Group 0 is the default group, the one the BIOS leaves selected. For any other group, `out_port` and `inp_port` select the group, do the input or output, and select group 0 again, as the BIOS does. For group 0 they leave the expander alone, so use group 0 on a system that has no expander.

The group is written to the expander port as it is, so it is the same number the BIOS uses for the group.

Ticks

The tick functions keep time with the real time clock of the 1802/Mini and the 1802/MAX. `tick_start` sets the clock to interrupt 64 times in a second, `TICKS_PER_SECOND`, and an interrupt routine in the library counts the interrupts.

`tick_wait` is for doing something at a steady rate. It waits until `n` ticks after the time the wait before it ended, not `n` ticks from when it is called, so the time between two waits does not depend on how long the work between them takes:

```
#include <gpio.h>

#define RTC  PORT(0, 1)

int main(void) {
  tick_start(RTC);

  while (!ef4_asserted()) {
    tick_wait(TICKS_PER_SECOND / 4);
    step();                              /* four times in a second */
  }

  tick_stop();
  return 0;
}
```

- If the work takes longer than `n` ticks, `tick_wait` does not wait, and the next wait is shorter to make up for it. If it takes twice `n` ticks or more, the program can not keep up, and the time is counted from then on.
- `ticks` is the count itself, a byte that starts over after 255. The difference of two counts is right as long as it is less than 256 ticks, which is four seconds.
- `tick_stop` puts the clock back as the BIOS has it and R1 as it was, and leaves interrupts disabled. Call it before the program ends.
- The ticks use the interrupt and R1, so a program can have no other interrupt routine while they run.

- Do not read or set the time of the clock while the ticks run, with `systime` or otherwise. The interrupt routine writes to the clock, and when it does so in the middle of the BIOS reading the time, the time that is read is wrong. This is why `ledclock.c` does not use the ticks.

I2C

The I2C functions are in two parts. The part in `gpio.lib` takes a message apart into starts, bytes and a stop, and is the same for all hardware. A driver does those on the hardware, and is a library of its own that the program is linked with. The program names the driver when it is built, and `gpio.h` only declares the I2C functions if it does:

```
elfc -D I2C_BB prog.c
```

See I2C drivers for the drivers there are.

`i2c_init` sets the port of the hardware and has to be called first. It frees the bus as `i2c_clear` does, and is true if the data line is high afterwards:

```
#include <gpio.h>

#define I2C_BUS  PORT(1, 3)
#define SENSOR   0x48

int main(void) {
  byte reg = 0x05;
  byte value[2];

  if (!i2c_init(I2C_BUS))
    return 1;

  if (!i2c_transfer(SENSOR, &reg, 1, value, 2))
    return 1;
  ...
```

The address is the 7 bit address of the device. The three message functions are true if the device acknowledged its address and every byte that was sent to it, and stop the message at the first byte that is not acknowledged.

- `i2c_write` sends a start, the address, `len` bytes and a stop.
- `i2c_read` does the same to receive `len` bytes. It acknowledges all of them but the last.
- `i2c_transfer` sends `wlen` bytes and then, after a repeated start, receives `rlen` bytes. This is how a register of a device is read: the bytes that are sent are the number of the register. A part with a length of 0 is left out, so it can also be used as `i2c_write` or `i2c_read`.
- With a length of 0, or both lengths 0, only the address is sent. The result shows if there is a device with that address.

I2C drivers

Symbol	Library	Hardware
`I2C_BB`	`i2c_bb.lib`	Two bits of an output port and two EF lines, bit-banged, as in the ELF-I2C library.

A program names its driver by defining the symbol, with the `-D` option of ElfC or with a `#define` before it includes `gpio.h`. There is no driver if none is named: the I2C functions are not declared then, and no driver library is needed.

```
elfc -D I2C_BB prog.c
```

If a program uses the I2C functions and no driver is named, the linker stops with `Symbol i2c_dinit not found`.

The program itself is the same for every driver, apart from the port it gives to `i2c_init`, which means what the driver wants it to mean.

For a driver that `gpio.h` does not know, define `I2C_OTHER_DRIVER` and name the library of the driver in the program:

```
#define I2C_OTHER_DRIVER
#pragma .link .library i2c_mine.lib
#include <gpio.h>
```

The bit-banged driver

Line	Output	Input
SDA, data	bit 7 of the port	EF3
SCL, clock	bit 6 of the port	EF4

Each output bit goes to an open collector or open drain driver that pulls the line low when the bit is high. The EF lines read the lines back. To use other bits or EF lines, change them in `drivers/i2c_bb.asm` and build the library again.

A device may hold the clock line low to get more time, and the driver waits for it. A message takes about 4000 clock cycles for a byte, which is a millisecond at 4 MHz.

Writing a driver

A driver is one source file in the `drivers` folder, and `make` builds a library with the same name from every file there. To give it a symbol, add it to `include/gpio.h` next to `I2C_BB`. It is a procedure with the name `i2c_driver` that has a label for each thing the I2C functions ask of it:

Label	Does
`i2c_dinit`	Set up the hardware for the port given to `i2c_init`.
`i2c_dopen`	Get ready for a message.
`i2c_dstart`	Make a start or a repeated start and send the address.
`i2c_dwrite`	Send a byte and get the acknowledge.
`i2c_dread`	Receive a byte and acknowledge it, or not if it is the last.
`i2c_dstop`	Make a stop.
`i2c_dclear`	Free the bus from a device that holds the data line low.

`src/i2c.inc` describes how they are called and which registers they may use, and `drivers/i2c_bb.asm` is a driver to start from.

SPI

The SPI functions are for the SPI adapter of the 1802/Mini and the 1802/MAX, which sends and receives a byte with a shift register. They are part of `gpio.lib` and need no driver.

`spi_init` sets the control port and the data port of the adapter, which have to be in the same group, and has to be called first. It writes nothing to the adapter.

`spi_control` sets the four lines of the adapter. The lines that are named are high, and a chip is selected when its line is named:

Line	
`SPI_CS0`, `SPI_CS1`	Chip select 0 and 1.
`SPI_OUT0`, `SPI_OUT1`	General output 0 and 1.

```
#include <gpio.h>

  byte id[3];
  byte cmd = 0x9f;

  spi_init(PORT(0, 2), PORT(0, 3));

  spi_control(SPI_CS0);
  spi_write(&cmd, 1);
  spi_read(id, 3);
  spi_control(0);
```

- `spi_write` sends `len` bytes.
- `spi_read` receives `len` bytes and sends `0xff` for each of them.
- `spi_transfer` sends `len` bytes and stores the byte that came in with each of them. The two pointers may be the same.

These three write the data port for every byte, which is right for the few bytes of a command or a register. For a lot of bytes, such as the image of a display, there are two that use the DMA of the adapter:

- `spi_write_dma` sends `len` bytes, as `spi_write` does.
- `spi_read_dma` receives `len` bytes, as `spi_read` does, but clocks one more byte out of the device than it stores, which is how the adapter receives with DMA. Use `spi_read` for a device that this is a problem for.

DMA takes one machine cycle for a byte, where the others take twelve, so 1024 bytes take 2.7 milliseconds at 4 MHz in place of 25. It takes longer to start, so up to about ten bytes it is no faster. The bytes are done 64 at a time, as the BIOS does, so interrupts are not held up for long.

On a system with a micro SD card, the operating system uses the same adapter for the card, on chip select 1. Do not leave a chip selected while calling the operating system, and expect the general outputs to be high afterwards.

Examples

`make examples` builds the programs in the `examples` folder. They get their ports from `include/sysconfig.h`, for the system given with `SYSTEM`, which is `1802MAX` if it is not given, and are linked with the I2C driver given with `I2C`, which is `BB`:

```
make -C examples SYSTEM=SUPERELF I2C=BB TARGET_OS=elfdos
```

Example	Description
`demo.c`	Counts on the LEDs until the input button is pressed, then shows the EF lines and the switches.
`led7/`	A library, `led7.lib`, for a four digit seven segment LED display with an HT16K33 on the I2C bus, such as the Adafruit 0.56" backpack. It is written in C, and `led7/led7.h` describes its functions.
`ledclock.c`	Shows the time on that display for a number of seconds, 10 if no number is given, and then turns the display off.
`ssd1305/`	A library, `ssd1305.lib`, for a 128 by 64 OLED display with an SSD1305 on the SPI adapter, as in Elf-Elfos-oled. It is written in C, and `ssd1305/ssd1305.h` describes its functions.
`show.c`	Shows an image on that display. The file is an image of 1024 bytes, or a Pixie image of 256 bytes, which is made twice as wide and as high. There are some in `images/`.
`play.c`	Plays a movie on that display, 32 images in a second, until the input button is pressed. The file is images of 1024 bytes. There are some in `movies/`.
`dispoff.c`	Clears that display and turns it off.

The LED display

The `led7` library keeps the digits and the colon in a buffer, and sends the buffer to the display in one message:

```
#include <gpio.h>
#include "led7/led7.h"

  i2c_init(PORT(1, 3));
  led7_init(LED7_ADDR);

  led7_write_digit(0, 1);
  led7_write_digit(1, 2);
  led7_write_colon(true);
  led7_write_digit(2, 3);
  led7_write_digit(3, 4);
  led7_write_disp();          /* shows 12:34 */
```

The OLED display

The `ssd1305` library shows an image of 1024 bytes, which a program reads from a file or draws itself. `ssd1305_init` resets the display and turns it on, and has to be called first, after `spi_init`:

```
#include <gpio.h>
#include "ssd1305/ssd1305.h"

  static byte image[SSD1305_FRAME_SIZE];

  spi_init(PORT(0, 2), PORT(0, 3));
  ssd1305_init();

  ssd1305_display(image);
  ...
  ssd1305_clear();
  ssd1305_sleep();            /* off, until ssd1305_wake */
```

Function	Description
`void ssd1305_init(void)`	Reset the display and turn it on.
`void ssd1305_display(const byte *frame)`	Show an image.
`void ssd1305_clear(void)`	Clear the image.
`void ssd1305_set_contrast(byte level)`	Set the contrast, 0 to 255.
`void ssd1305_sleep(void)`	Turn the display off. It keeps its image.
`void ssd1305_wake(void)`	Turn the display on again.
`void ssd1305_pixie(const byte *pixie, byte *frame)`	Make an image from a Pixie image of 256 bytes.

An image is the eight rows of eight pixels from the top down, and each row is a byte for each column from left to right, with the top pixel of the eight in bit 0.

These are the images in `images/` to try `show` with:

Image	
`hires.img`	Higher resolution drawing of the starship Enterprise.
`imp.img`	Graphics logo featuring imp character.
`blinky.img`	Sample PAC-MAN game board.
`starship.img`	Classic Pixie graphics starship image.
`mona.img`	The Mona Lisa in black and white dots. It is a portrait, upright when the display is turned so that its left edge is up.

`mkimage.py` makes an image like `mona.img` from any picture, and needs the Pillow package of Python.

These are the movies in `movies/` to try `play` with. Three of them are made by a Python script, which can be changed to make others:

Movie	Script	
`sample1.mov`		The movie of Elf-Elfos-oled.
`walk.mov`	`mkwalk.py`	A stick figure that walks, one second long.
`globe.mov`	`mkglobe.py`	The earth turning, three seconds long. The land is from the public domain map data of Natural Earth.
`trench.mov`	`mktrench.py`	A wire frame attack on a battle station, after the briefing room display of Star Wars, seven seconds long.

`play.c` takes its time from the tick functions, two ticks for an image. A C program is slower than one in assembly: next to reading the file, an image takes about 10 milliseconds at 4 MHz, of which 3 are sending it to the display. If reading 1024 bytes takes more than what is left of the 31 milliseconds for an image, the movie plays slower, as fast as it can be read.

Notes

- An EF line is asserted when its pin is low, which is when the `B1` to `B4` instructions branch.
- The port index is part of the opcode of the 1802 input and output instructions, so `out_port`, `inp_port` and `set_exp_port` store the instruction they need into the library code before it runs. The program has to run from RAM, as programs for Elf/OS and ELF-DOS do, and these functions must not be called from an interrupt routine. The I2C, SPI and tick functions do the same when they are given their port.
- Only the low three bits of the port index are used. Do not use index 0, there is no such port.
- An interrupt routine that does I/O of its own while another group is selected will use the wrong group.
- `spi_write_dma` and `spi_read_dma` use R0, the DMA pointer, and leave it after the last byte.

- The bit-banged I2C driver writes the other six bits of its port as 0, and waits for as long as the clock line is low, so a message does not end if that line is held low.
- The bit-banged I2C driver, the SPI functions and the tick functions select the port group with the port that is set when `i2c_init`, `spi_init` or `tick_start` is called, so call `set_exp_port` before them.
- The delay functions read the clock frequency in kHz from memory locations `$3E` and `$3F`, high byte first, where EDOS-mbios and other BIOSes store it. If both are zero, `FREQ_KHZ` is used.
- The delay functions count the time it takes to call them from C as part of the delay, and carry the fraction of a loop that is left over into the next millisecond, so the error does not add up. A delay is within about 120 clock cycles of the time asked for, which is 30 microseconds at 4 MHz. The shortest delay is the time the call itself takes, about half a millisecond at 4 MHz and one millisecond at 1.79 MHz.
- A delay is longer by the time spent in interrupts and DMA while it runs.
- The clock has to be 300 kHz or faster for the delay functions.